

LC 1695: Maximum Erasure Value

FreeCash

Amazon

Problem Statement: Choose a subarray with all unique elements and return the maximum possible score of that subarray.

Approach & Core Logic

- **State Detection:** When a duplicate appears, the current window becomes *invalid*.
- **Resolution Action:** Shrink the window from the left until the duplicate disappears.
- **Validation Rule:** A window is invalid if `frequency[current_number] > 1`.
- **Continuation:** Once valid, continue expanding the window to the right.

Complexity Analysis

Brute Force Approach ($O(n^2)$)	Optimal Approach (Sliding Window)
<pre>for(int i=0; i<n; i++) { unordered_set<int> st; int sum = 0; for(int j=i; j<n; j++) { if(st.count(nums[j])) { break; } st.insert(nums[j]); sum += nums[j]; ans = max(ans, sum); } }</pre> return ans;	<pre>while(r < n) { score += nums[r]; mp[nums[r]]++; while(mp[nums[r]] > 1) { mp[nums[l]]--; score -= nums[l]; l++; } ans = max(ans, score); r++; }</pre> return ans;

Complete C++ Solution

This optimal solution implements the sliding window technique using a hash map to maintain the frequency of elements in the current valid subarray.

```
class Solution {
public:
    int maximumUniqueSubarray(vector<int>& nums) {
        int n = nums.size();
        int ans = 0;
        int score = 0;
        int r = 0;
        int l = 0;
        unordered_map<int, int> mp;

        while (r < n) {
            score += nums[r];
            mp[nums[r]]++;

            // Shrink window from the left until it is valid again
            while (mp[nums[r]] > 1) {
                mp[nums[l]]--;
                score -= nums[l];
                l++;
            }

            ans = max(ans, score);
            r++;
        }

        return ans;
    }
};
```